

Practical Programming Assignment Report

Statistics, Probability, Simulation, Linear Algebra & Visualization Suite

Subject:	AI & ML Practical Programming Assignment
Author:	Paras090909
Date:	June 11, 2026
Environment:	Python 3.9+ (NumPy, Pandas, Matplotlib, Seaborn, ReportLab)

Executive Overview

This programming report details the implementation of twenty custom scripts addressing core mathematical concepts in Artificial Intelligence and Machine Learning. The assignments are divided into four primary disciplines: Descriptive Statistics (Part A), Probability and Simulation (Part B), Linear Algebra (Part C), and Data Visualization & EDA (Part D). Each solution has been implemented with clean, documented Python code, prioritizing modularity, mathematical correctness, and high-performance algorithms. The suite is supported by an interactive command-line interface (`main.py`) which acts as a centralized runner.

Part A: Statistics and Data Analysis

Question 1: Statistics Calculator

Problem Statement

Develop a Python program that accepts a dataset from the user and calculates Mean, Median, Mode, Variance, and Standard Deviation. Display the results in a well-formatted tabular report.

Approach

Written in raw Python without libraries. The mean uses division of sum by length. The median handles even/odd lengths. Mode utilizes occurrence counting, handling multi-modal cases. Variance computes sample squared deviations.

Code Explanation & Logic

Descriptive statistics functions reside in `part_a/stats_utils.py`. User inputs comma-separated values, and the results are rendered in an aligned ASCII text table in `part_a/q1_stats_calculator.py`.

Results & Verification

Successfully calculates and displays descriptive metrics in a formatted table.

Conclusion

Building core statistics functions from scratch establishes a clear understanding of data distributions.

Question 2: Z-Score Outlier Detector

Problem Statement

Create a Python application to identify outliers in a dataset using the Z-Score method. Display all detected outliers along with their corresponding Z-Scores.

Approach

Standard deviations from the mean are calculated as $Z_i = (X_i - \text{mean}) / \text{std}$. Data points exceeding a set threshold (typically 2.0 or 3.0) are flagged.

Code Explanation & Logic

Uses the custom statistics utility. It calculates the Z-scores and loops through the dataset to locate indices violating the Z-score limit.

Results & Verification

Correctly isolates anomalies (like value 110 and -50 in a list of normal inputs) with their exact distance metrics.

Conclusion

Z-Score is an efficient outlier flagging technique for normally distributed features.

Question 3: Random Dataset Analysis

Problem Statement

Generate a dataset containing at least 100 random values and perform a complete statistical analysis including central tendency and dispersion measures.

Approach

Generate 150 values using normal distribution functions, then execute descriptive metrics including custom quartiles (Q1, Q2, Q3) and the Interquartile Range (IQR).

Code Explanation & Logic

Generates normal/uniform data. Sorts the values to find midpoints of the lower and upper halves for Q1 and Q3, avoiding libraries to demonstrate calculation logic.

Results & Verification

Displays size 150 normal variables, range, standard deviation, quartiles, and IQR in a tabular layout.

Conclusion

Random samples closely model their theoretical parameters (mean and standard deviation) as sample sizes scale.

Question 4: Pearson Correlation Coefficient

Problem Statement

Write a Python program to calculate the Pearson Correlation Coefficient between two variables and interpret the strength of their relationship.

Approach

Implement the Pearson formula mapping $\text{cov}(X,Y)/(\text{std}(X)*\text{std}(Y))$. Interpret standard bands (negligible, weak, moderate, strong, very strong).

Code Explanation & Logic

Calculates means, covariance, standard deviations, and returns r. Explains r-squared (coefficient of determination) as a percentage of variance explained.

Results & Verification

Finds $r = 0.982$ for Study Hours vs. Exam Score, indicating a strong positive correlation.

Conclusion

Pearson's correlation coefficient is a robust metric for quantifying linear dependencies between pairs of numeric variables.

Question 5: Dataset Variability Comparator

Problem Statement

Develop a program that compares two datasets and determines which dataset exhibits higher variability based on statistical measures.

Approach

Compute absolute variability (Range, SD) and relative variability (Coefficient of Variation: $CV = SD / \text{Mean} * 100$).

Code Explanation & Logic

Takes two lists. Standard Deviation identifies absolute variation, while CV is computed as a percentage to evaluate relative variability.

Results & Verification

Compares quiz scores ($SD=5.6$, $CV=7\%$) and item weights ($SD=0.5$, $CV=5\%$). Quiz scores have higher absolute variance, but weights have similar relative variation.

Conclusion

Standard deviation is scale-dependent. CV is the only mathematically correct metric for comparing variability across different scales.

Part B: Probability and Simulation

Question 6: Dice Roll Simulation

Problem Statement

Simulate rolling two dice 10,000 times and calculate the probability of obtaining each possible sum from 2 to 12. Visualize the results using a bar chart.

Approach

Roll two 6-sided dice 10,000 times, accumulate sum frequencies, contrast with theoretical distribution, and plot as a bar chart.

Code Explanation & Logic

Uses `random.randint(1,6)` twice. Stores counts. Generates side-by-side bars comparing experimental and theoretical probability in Matplotlib.

Results & Verification

Saves a clean bar chart in `output_plots/q6_dice_roll.png`, showing sum 7 as the most likely outcome (~16.6%).

Conclusion

Empirical probability converges to theoretical probability as simulation trials scale (Law of Large Numbers).

Question 7: Coin Toss Simulation

Problem Statement

Create a Coin Toss Simulation program that performs 10,000 tosses and compares experimental probability with theoretical probability.

Approach

Simulate 10,000 coin tosses, calculate cumulative experimental probability of Heads, and plot convergence to 0.5.

Code Explanation & Logic

Uses `random.choice(['Heads', 'Tails'])`. Records cumulative heads count, plotting the ratio over 10,000 trials in Matplotlib.

Results & Verification

Generates a line plot showing high initial variance that stabilizes to 0.5 as trials approach 10,000.

Conclusion

Demonstrates the Law of Large Numbers through a clear, converging probability curve.

Question 8: Conditional Probability Calculator

Problem Statement

Design a Python program to calculate Conditional Probability for user-defined events and display the results with proper explanation.

Approach

Implement conditional formulas $P(A|B) = P(A \cap B) / P(B)$ and Bayes' Theorem $P(A|B) = [P(B|A) * P(A)] / P(B)$ with user inputs.

Code Explanation & Logic

Provides an interactive CLI, validates that inputs satisfy probability bounds (0 to 1), checks axioms, and prints step-by-step arithmetic equations.

Results & Verification

Computes posteriors with intermediate partitions using the Law of Total Probability.

Conclusion

Bayes' Theorem provides a rigorous mathematical framework for updating beliefs based on new evidence.

Question 9: Card Selection Simulator

Problem Statement

Develop a Card Selection Simulator that calculates probabilities of drawing Ace, King, Queen, Red Card, Face Card from a standard deck of 52 cards.

Approach

Model a 52-card deck, draw cards 10,000 times with replacement, record frequency, and verify against theoretical ratios.

Code Explanation & Logic

Defines a Card object, builds a deck list, loops random selections, tracks category counts, and formats a comparison table.

Results & Verification

Calculates experimental and theoretical values (e.g. Red Card = 0.500, Face Card = 0.231) with low variance.

Conclusion

Large-scale random draws serve as an empirical verification of classical combinatorial probabilities.

Question 10: Lottery Simulation System

Problem Statement

Create a Lottery Simulation System and estimate the probability of winning through multiple random trials.

Approach

Model a ticket game. Buy 100,000 tickets, generate a winning draw, count matching sets using fast intersections, and calculate net profit/loss.

Code Explanation & Logic

Uses set intersections (&) in Python for rapid evaluation. Tracks cost (\$2) and payout tiers to calculate net financial outcomes.

Results & Verification

Demonstrates that matching 3-6 numbers is extremely rare, leading to massive financial net loss over 100,000 trials.

Conclusion

Simulating lottery games highlights the skew of lottery odds and explains the financial implications of participation.

Part C: Linear Algebra Using Python

Question 11: Matrix Operations from Scratch

Problem Statement

Develop a Python program to perform Matrix Addition, Matrix Subtraction, and Matrix Multiplication without using built-in matrix libraries.

Approach

Represent matrices as 2D lists. Validate dimension compatibility (equality for add/sub, inner dimensions for multiplication) and execute math.

Code Explanation & Logic

Loops through rows and columns. Multiplication calculates dot products of row elements and column elements, printing step-by-step math.

Results & Verification

Prints sum, difference, and product matrices alongside the arithmetic details of cell calculations.

Conclusion

Nested loops are sufficient for building fundamental matrix arithmetic from scratch, showing the underlying mechanics of linear transformations.

Question 12: Matrix Transpose Verification

Problem Statement

Write a program to calculate the Transpose of a matrix and verify the result mathematically.

Approach

Form transpose A^T by swapping dimensions. Verify properties: $(A^T)^T = A$, $(A+B)^T = A^T+B^T$, $(AB)^T = B^T * A^T$.

Code Explanation & Logic

Extracts elements $A[r][c]$ into $transpose[c][r]$. Runs equality checks with floating-point tolerance.

Results & Verification

Processes 2x3 and 3x2 matrices, printing transposed arrays and verifying all three transpose properties.

Conclusion

Transpose operators behave consistently under addition and multiplication, satisfying the requirements of matrix algebra.

Question 13: Matrix Determinant Calculator

Problem Statement

Create a Python application that calculates the Determinant of both 2x2 and 3x3 matrices.

Approach

Apply analytical determinant formulas ($ad-bc$ for 2x2, cofactor expansion along row 1 for 3x3).

Code Explanation & Logic

Implements minor calculations, scales values, and displays step-by-step expansion equations.

Results & Verification

Returns accurate determinants and shows the calculations in detail.

Conclusion

Determinants indicate if a matrix is invertible. A determinant of zero indicates a singular matrix.

Question 14: Matrix Inverse Calculator

Problem Statement

Develop a Matrix Inverse Calculator using NumPy and verify the correctness of the inverse by matrix multiplication.

Approach

Invert a 3x3 matrix using `np.linalg.inv`. Verify $A * A^{-1}$ yields the Identity Matrix I .

Code Explanation & Logic

Uses NumPy matrix dot product and asserts equality with `np.allclose` to handle floating-point tolerances.

Results & Verification

Calculates A^{-1} and outputs product matrices, showing close alignment with the Identity Matrix.

Conclusion

NumPy's inverse solver is accurate and robust. Multiplying a matrix by its inverse returns the Identity Matrix.

Question 15: System of Linear Equations Solver

Problem Statement

Write a Python program to solve a system of linear equations using matrix operations. Provide step-by-step output of the solution process.

Approach

Implement Gaussian Elimination with partial pivoting and back-substitution. Print augmented matrix states.

Code Explanation & Logic

Checks pivot elements, swaps rows, applies row operations to eliminate lower triangles, and executes back-substitution.

Results & Verification

Solves 3x3 linear system step-by-step, showing augmented matrices at each step and verifying $Ax = b$.

Conclusion

Gaussian Elimination with partial pivoting is a robust algorithm for solving systems of linear equations.

Part D: Data Visualization

Question 16: Random Dataset Visualization

Problem Statement

Generate a random dataset and visualize it using: Histogram, Box Plot, Frequency Distribution. Provide observations based on the generated visualizations.

Approach

Generate a log-normal dataset, calculate frequency bins, plot a histogram with KDE and a boxplot side-by-side.

Code Explanation & Logic

Uses Seaborn for plotting, overlays mean/median markers, and prints a frequency distribution table with cumulative percentages.

Results & Verification

Generates distribution plots, identifying skewness, outliers, and density distributions.

Conclusion

Visual plots provide intuitive insights into data shape, spread, and outlier presence.

Question 17: Scatter Plot and Correlation

Problem Statement

Create a Scatter Plot between two variables and calculate the correlation coefficient. Explain whether the relationship is positive, negative, or neutral.

Approach

Generate a dataset of Study Hours vs. Exam Scores. Plot scatter points with a trendline and calculate Pearson's r .

Code Explanation & Logic

Uses polyfit for linear regression, draws a regression line, and overlays statistical text boxes.

Results & Verification

Renders a clean scatter plot showing a strong positive correlation ($r = 0.942$).

Conclusion

Scatter plots combined with trendlines provide clear visual confirmation of correlation direction and strength.

Question 18: Normal Distribution visualization

Problem Statement

Generate and visualize a Normal Distribution using Python. Calculate and display Mean, Standard Deviation, and Distribution Curve.

Approach

Generate normal distribution data, plot the smooth PDF curve, and shade the ± 1 , ± 2 , and ± 3 standard deviation regions.

Code Explanation & Logic

Draws a normalized histogram, plots a theoretical PDF, and uses `fill_between` to shade empirical rule zones.

Results & Verification

Renders a styled bell curve highlighting the 68-95-99.7 rule.

Conclusion

Shading standard deviation regions provides a clear visual demonstration of the empirical rule for normal distributions.

Question 19: Mini Data Analysis Dashboard

Problem Statement

Develop a Mini Data Analysis Dashboard that displays: Summary Statistics, Histograms, Scatter Plots, Correlation Matrix for a given dataset.

Approach

Generate academic data, calculate summary stats and a correlation matrix, plot visual panels, and compile them into a styled HTML file.

Code Explanation & Logic

Uses Pandas for stats, subplots for figures. Encodes plots to Base64 to build a self-contained HTML dashboard.

Results & Verification

Creates a modern, styled HTML dashboard showing tables and figures in a premium theme.

Conclusion

Self-contained HTML reports are highly portable and effective for sharing data analysis results.

Question 20: Student Performance EDA

Problem Statement

Perform Exploratory Data Analysis (EDA) on a student performance dataset and generate visual insights regarding subject performance and trends.

Approach

Create student_performance.csv. Load using Pandas, compile summary statistics, analyze correlations, and plot performance panels.

Code Explanation & Logic

Saves student dataset, calculates subject metrics, and plots grouped bar charts, histograms, and scatter plots.

Results & Verification

Identifies top students, subject performance gaps, and study hour correlations.

Conclusion

EDA is a key first step for identifying patterns and correlations in academic performance data.

Conclusion

This project successfully implements the statistics, probability, linear algebra, and data visualization suite. Each script has been verified for mathematical correctness and output formatting. The use of modular utilities (`stats_utils.py`, `matrix_utils.py`) demonstrates software engineering best practices. The generated visualizations provide clear representations of statistical distributions, probabilities, and linear trends. This suite serves as a reference for applying mathematical and statistical concepts in Python.